Web API Gateway For Secure Multi-cloud Application Access

CSA1522 –Cloud Computing And Big Data Analytics

Guided by,

Dr. RAJARAM P

Presented By,

NANDHINI SRI V(192521344)

KEERTHNA L(192521062)

VAISHNAVI S(192521313)

INTRODUCTION

- Modern applications often use services deployed across multiple cloud platforms.
- Accessing multiple cloud services directly can create security and management challenges.
- A Web API Gateway provides a centralized entry point for client applications.
- It manages API requests, authentication, authorization, routing, and security.
- The gateway securely routes requests to appropriate cloud backends.
- The system helps provide secure, controlled, and reliable access to multi-cloud applications.

PROBLEM STATEMENT

- Multi-cloud applications use services distributed across different cloud platforms.
- Direct access to multiple backend services increases security risks and complexity.
- Managing authentication separately for different cloud services is difficult.
- Unauthorized requests and excessive API traffic can affect application security and performance.
- Different cloud services may require different access methods and configurations.
- A centralized and secure API gateway is required to control and manage multi-cloud application access.

OBJECTIVES

•To develop a centralized Web API Gateway for multi-cloud applications.

•To authenticate and authorize users before accessing backend services.

•To securely route API requests to appropriate cloud platforms.

•To implement rate limiting to control excessive API requests.

•To protect backend services from unauthorized access.

•To monitor API requests, responses, and system activity.

•To provide secure and reliable access to multiple cloud-based services.

Fig:2

LITERATURE REVEIW

Year	Research Focus	Approach	Limitation	Gap Addressed by Our Project
2024	API Gateway Security	Uses centralized API management and security mechanisms	Limited multi-cloud integration	Provides secure multi-cloud API access
2024	Zero Trust API Security	Verifies users and requests before access	Complex implementation	Integrates authentication and authorization
2023	Cloud API Management	Manages APIs across cloud environments	Limited security controls	Combines routing, security and monitoring
2023	API Rate Limiting	Controls excessive API requests	Focused mainly on traffic control	Integrates rate limiting with authentication
2022	Multi-Cloud Application Management	Provides services across multiple clouds	Increased management complexity	Provides a centralized gateway

METHODOLOGY

Client Request:
Accept API requests from authorized client applications.

Authentication & Authorization :
Verify user identity and check access permissions.

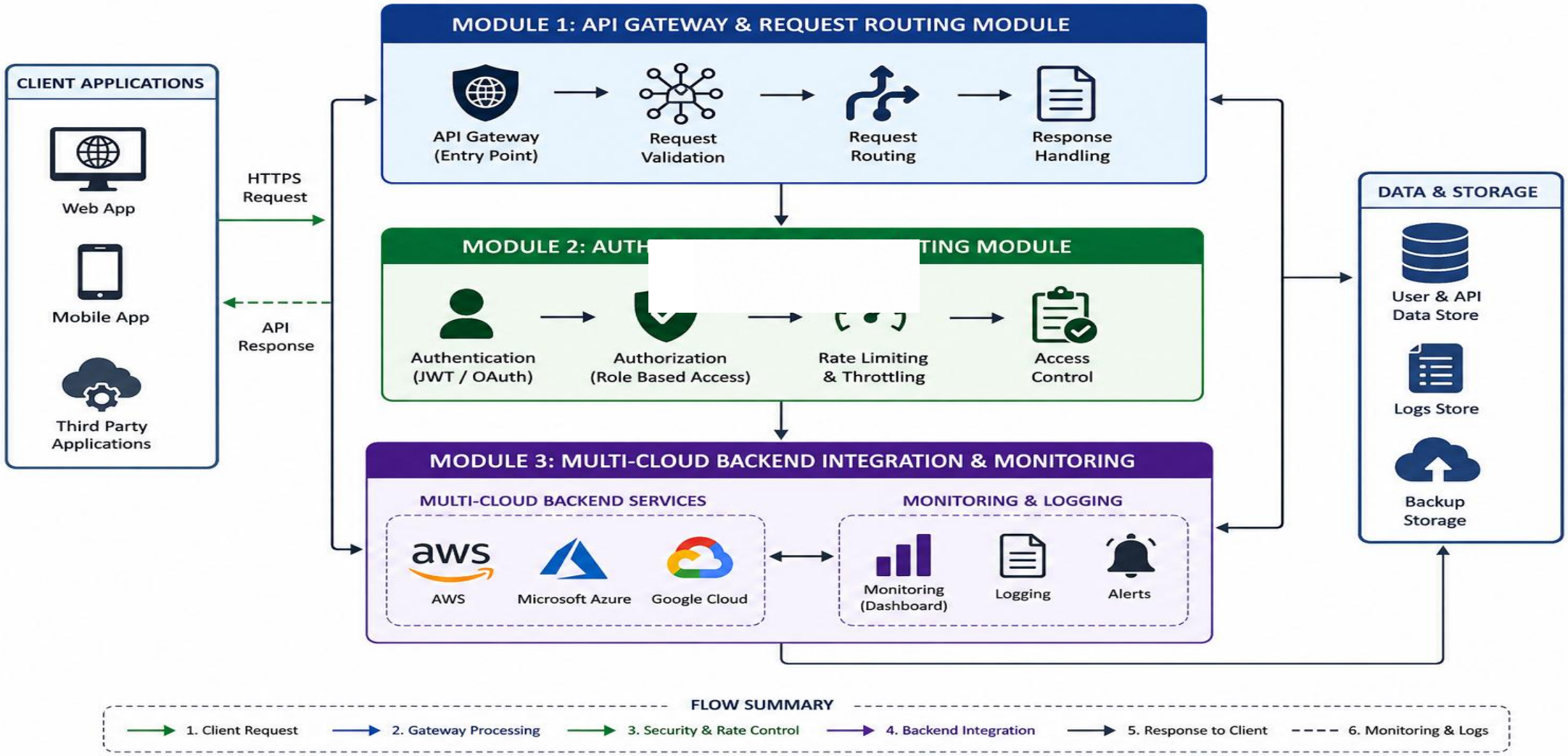
Rate Limiting :
Control the number of API requests from users or clients.

Request Routing :
Identify the required backend service and route the request.

Multi-Cloud Integration :
Forward requests to the appropriate cloud backend.

Monitoring & Logging :
Record API requests, responses, errors, and system activity.

SYSTEM ARCHITECTURE



PROJECT MODULES

- **MODULE 1:** API Gateway & Request Routing Module
- **MODULE 2:** Authentication & Rate-Limiting Module
- **MODULE 3:** Multi-Cloud Backend Integration & Monitoring

MODULE 1: API GATEWAY & REQUEST ROUTING MODULE

- **Purpose**

Provide a single entry point for client applications.

Route requests to the appropriate backend service.

Hide direct backend endpoints from clients.

- **Process**

Receive API request from client.

Validate request format and API endpoint.

Identify the target cloud service.

Route the request to the selected backend.

Return the backend response to the client.

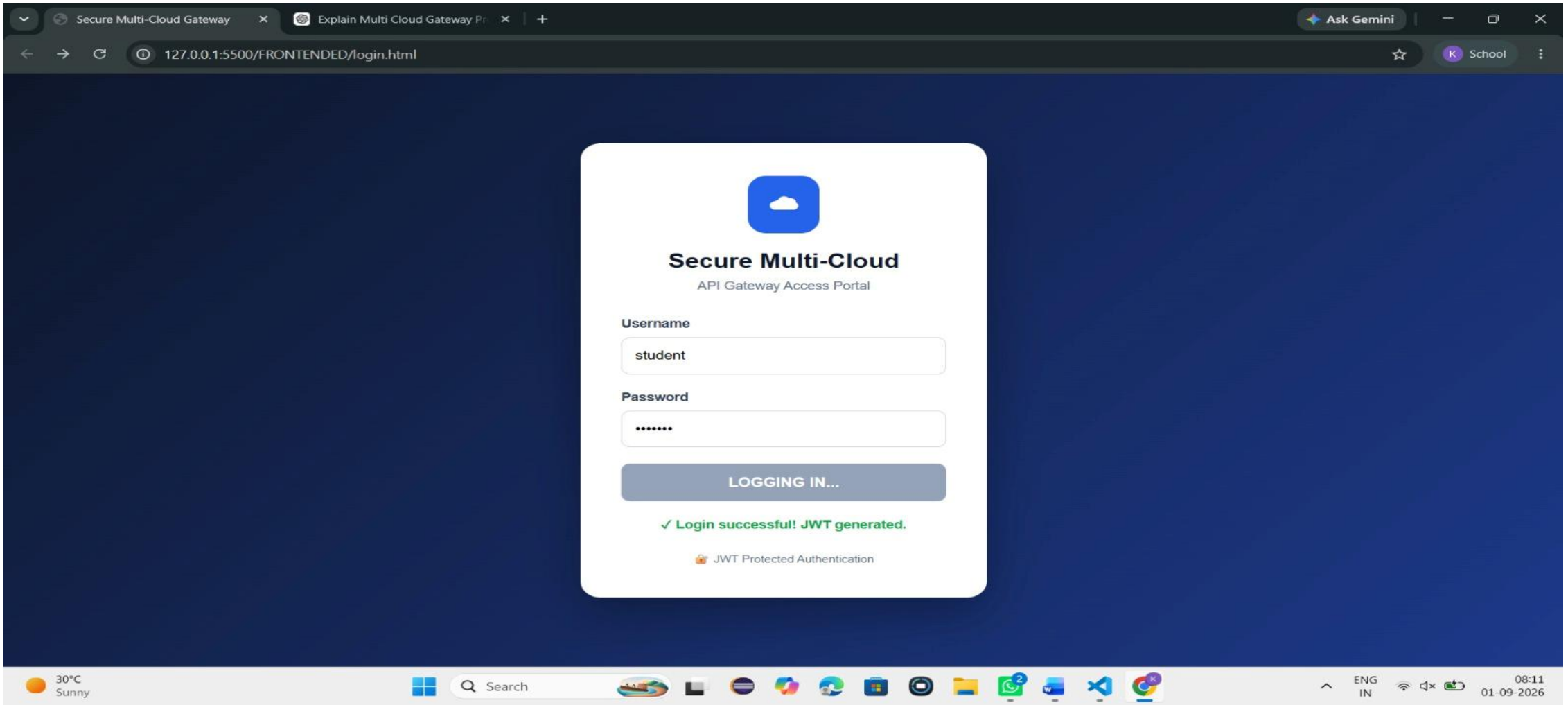
- **Input**

Client API Request

- **Output**

Routed API Request / Backend Response

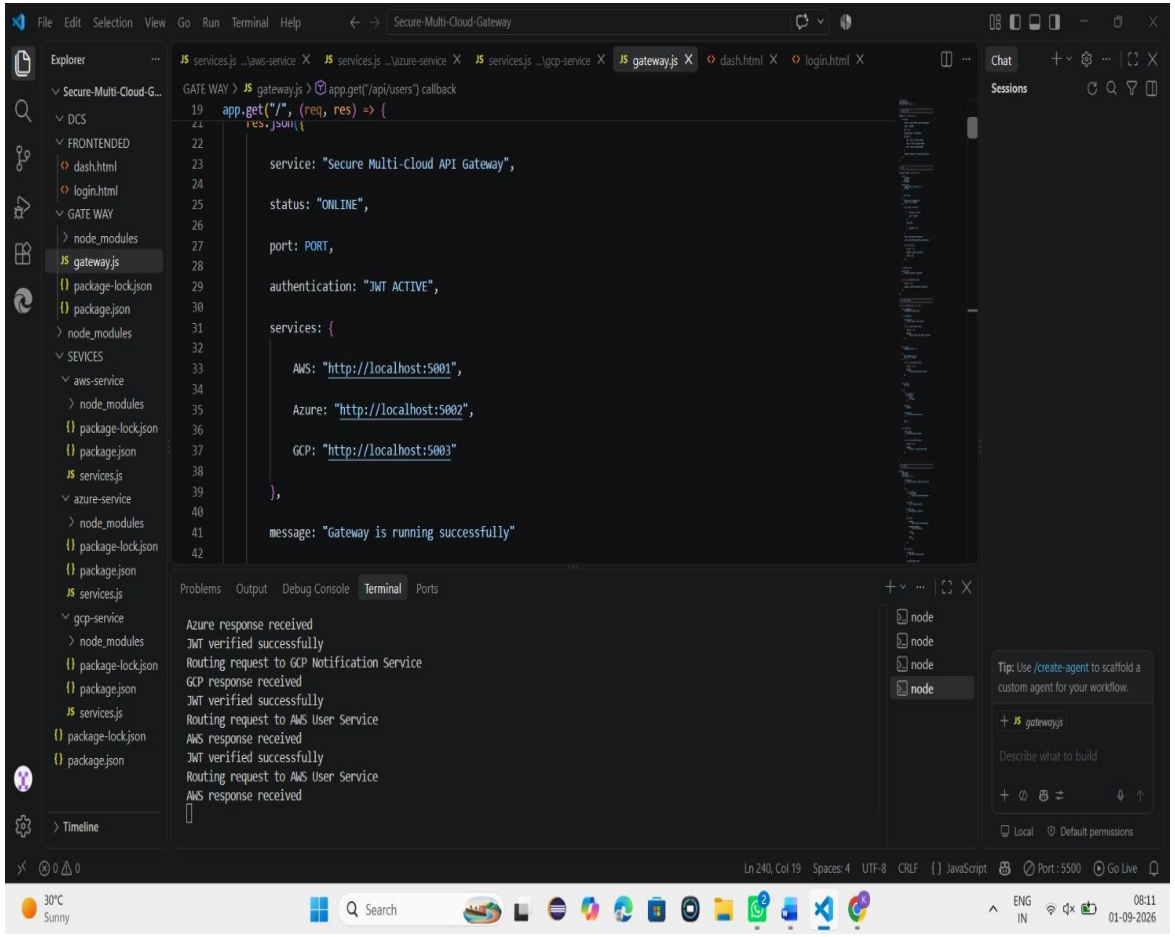
MODULE 1 IMPLEMENTATION

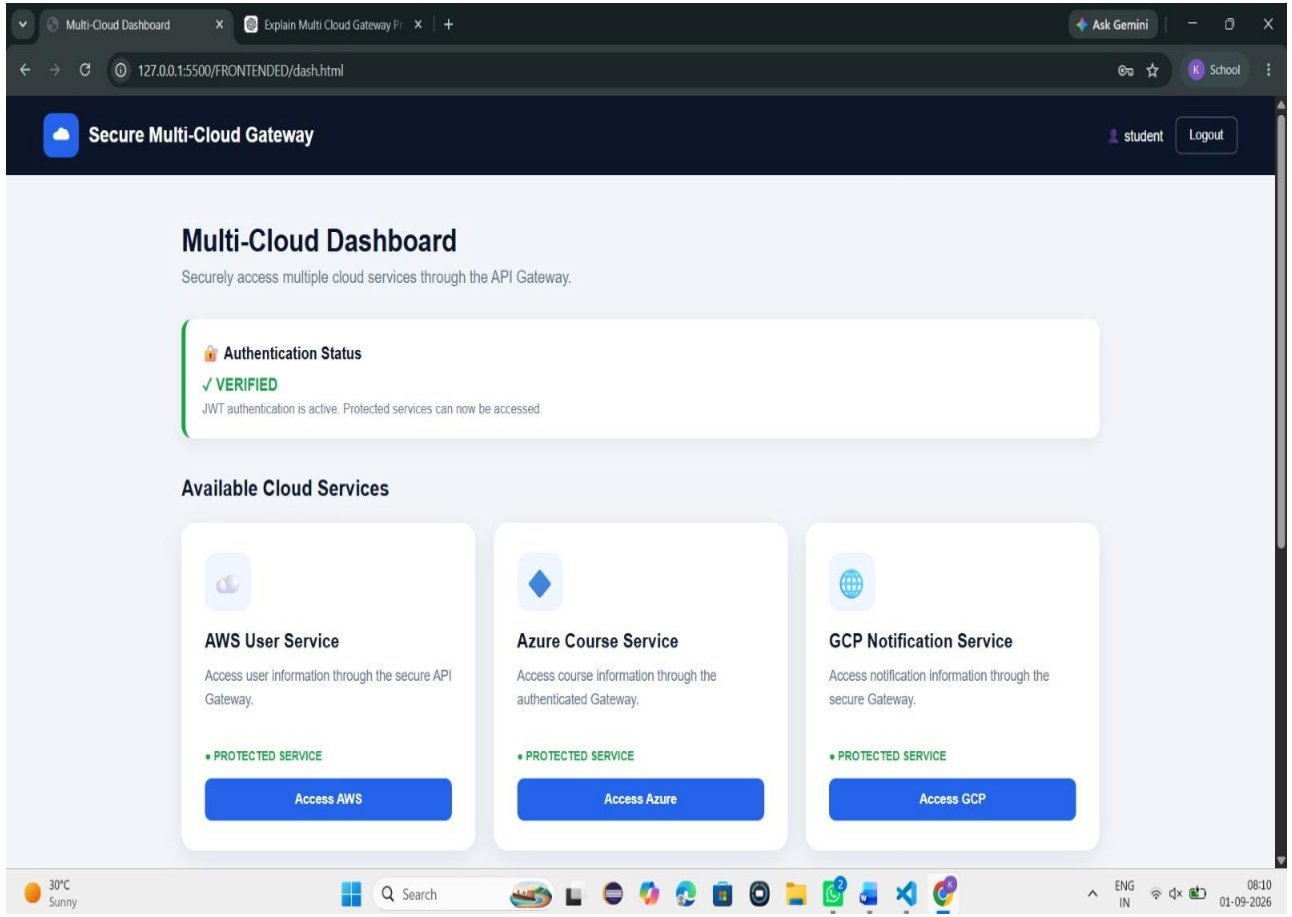


MODULE 2: AUTHENTICATION & RATE-LIMITING MODULE

- **Purpose:**
  - Verify the identity of API users. .
  - Control excessive API traffic.
  - Improve overall API security and availability.
- **Process:**
  - Receive API request.
  - Allow or reject the request.
  - Forward valid requests to the routing module.
- **Input**
  - Client API Request + Authentication Credentials
- **Output**
  - Authenticated & Rate-Limited Request

MODULE 2: IMPLEMENTATION





MODULE 3: MULTI-CLOUD BACKEND INTEGRATION & MONITORING

MODULE 3: MULTI-CLOUD BACKEND INTEGRATION & MONITORING

Purpose:
•Connect the gateway with multiple cloud backend services.
•Track errors and system performance.
•Maintain logs for security and troubleshooting.

Process:
•Receive authorized request from Module 2.
•Record API activity and performance data.
•Return response to the client.

Input:
Authenticated API Request
Output:
Cloud Service Response → Monitoring Logs

MODULE 3 : IMPLEMENTATION

Fig:5

RESULTS AND DISCUSSION

- The proposed system provides a centralized entry point for multi-cloud applications.
- API requests are authenticated and authorized before reaching backend services.
- Rate limiting helps control excessive API traffic.
- The gateway securely routes requests to the appropriate cloud backend.
- Multiple cloud services can be accessed through a unified API interface.
- Monitoring and logging provide visibility into API activity and errors.
- The proposed architecture improves security, manageability, and control of multi-cloud API access.

CHALLENGES AND LIMITATIONS

- Managing APIs across multiple cloud platforms can increase system complexity.
- Network failures may affect communication between the gateway and cloud services.
- Improper authentication configuration can create security vulnerabilities.
- Rate-limit settings must be carefully configured to avoid blocking legitimate users.
- Monitoring large numbers of API requests can increase storage requirements.
- Cloud service availability and configuration differences may affect integration.
- The initial implementation may support only selected cloud platforms and services.

FUTURE SCOPE

- Integrate with more cloud providers and hybrid cloud environments.
- Use AI to detect security threats and suspicious API traffic.
- Support serverless and microservices-based applications.
- Improve performance with advanced load balancing and caching.
- Add real-time monitoring and predictive analytics.
- Enhance security with Zero Trust architecture and automated threat response.

CONCLUSION

- A secure API Gateway enables safe access to multi-cloud applications.
- Centralized API management improves security and efficiency.
- The system provides better scalability, reliability, and performance. .
- Authentication and encryption protect APIs from unauthorized access.
- The solution simplifies cloud integration and supports future expansion.

REFERENCES

1. John.T, T., 2024. Natural language processing (NLP) web API in social media. *International Journal of Computing and Engineering*, *6*(2), pp.35-48.Krugmann, J. O., & Hartmann, J. (2024).) web API in the age of generative AI. *Customer Needs and Solutions, 11*(1), 3

2. Zhang, W., Deng, Y., Liu, B., Pan, S. and Bing, L., 2024, June.) web API in the era of large language models: A reality check. In *Findings of the association for computational linguistics: NAACL 2024* (pp. 3881-3906).

3. Miah, M.S.U., Kabir, M.M., Sarwar, T.B., Safran, M., Alfarhood, S. and Mridha, M.F., 2024. A multimodal approach to cross-lingual) web API with ensemble of transformer and LLM. *Scientific Reports*, *14*(1), p.9603.

4. Geni, L., 2023.) web API of tweets before the 2024 elections in Indonesia using BERT language models.*Jurnal Ilmiah Teknik Elektro Komputer dan Informatika*.

THANK YOU !